



武汉大学
WUHAN UNIVERSITY

金融数据分析作业：

《交通银行投资价值分析报告》

指导老师：胡利琴

2025 级 MF 班 彭俊铭 2025281050216

2025 年 12 月

《交通银行投资价值分析报告》

2025 级 MF 班 彭俊铭 2025281050216

一、交通银行背景介绍

交通银行始建于 1908 年，是中国历史最悠久的银行之一。1987 年 4 月 1 日，该行重新组建后正式对外营业，成为中国第一家全国性国有股份制商业银行，总部设在上海。2005 年 6 月在香港联交所挂牌上市，2007 年 5 月在上交所挂牌上市，2023 年入选全球系统重要性银行。按一级资本排名，该行居全球银行第 9 位。

交通银行以“建设具有特色优势的世界一流银行集团”为目标，以绿色为全集团业务经营发展底色，着力打造普惠金融、贸易金融、科技金融、财富金融四大业务特色，做实做细“五篇大文章”，持续提升客户经营、科技引领、风险管理、协同作战、资源配置五大专业能力，以“上海主场”建设、数字化转型的创新突破引领全行高质量发展。

经金融监管总局批准，交通银行通过手机银行、网上银行等线上服务渠道，以及境内 2,800 余家网点、境外 24 家分（子）行及代表处，为 295 万公司客户和 2.02 亿零售客户提供综合金融服务，包括存贷款、产业链金融、现金管理、国际结算与贸易融资、投资银行、资产托管、财富管理、银行卡、私人银行、资金业务等。本集团通过全资或控股子公司，涉足金融租赁、基金、理财、信托、保险、境外证券和债转

股等业务领域。

交通银行作为一家历史悠久的国有大型银行集团，将充分发挥国有大行服务实体经济主力军和维护金融稳定压舱石作用，保持战略定力，防范化解风险，努力为客户提供优质金融服务，为股东创造更多价值，为员工营造幸福家园，为社会做出更大贡献。

二、交通银行市场表现分析

1.股票价格获取。

本报告通过 AKShare 库工具，获取交通银行（A 股股票代码 601328）2020 年至 2024 年间共计五年间的 A 股交易数据，具体包括每日开盘价、收盘价、最高价、最低价、成交量、成交额、涨跌幅等关键数据，覆盖 2020 年-2024 年共计五年的市场周期，能够充分交通银行股票在该期间的市场表现。

2.交通银行分析代码。

```
import pandas as pd
import numpy as np
import akshare as ak
import pdfplumber
import os
import re
from collections import Counter
import jieba
import matplotlib.pyplot as plt
from wordcloud import WordCloud
from collections import Counter
```

```

# 导入可视化库
import matplotlib.pyplot as plt
import seaborn as sns
# 设置中文字体
plt.rcParams['font.family'] = 'sans-serif' # 全局字体类别
plt.rcParams['font.sans-serif'] = ['SimHei'] # 具体字体：黑体
plt.rcParams['axes.unicode_minus'] = False # 解决负号无法显示的问题

```

第一步：导入股票数据、财务数据(三张表)、核心宏观数据

```

# 从新浪接口获取某只股票的历史数据（包含日期）
# 基本参数，包括日期范围和股票代码
stock_code = "601328" # 交通银行 A 股
start_date = "20200101"
end_date = "20241231"
stock_code = "601328" # 交通银行
hist_df = ak.stock_zh_a_hist(symbol=stock_code,
period="daily", start_date=start_date, end_date=end_date)
print("历史数据列名:", hist_df.columns.tolist())
print("\n 历史数据前 5 行:")
print(hist_df.head())
# 保存为 CSV 文件
hist_df.to_csv('annual_reports/stock_data.csv', index=False,
encoding='utf-8-sig')

```

历史数据列名: ['日期', '股票代码', '开盘', '收盘', '最高', '最低', '成交量', '成交额', '振幅', '涨跌幅', '涨跌额', '换手率']

历史数据前 5 行:

	日期	股票代码	开盘	收盘	最高	最低	成交量	成交额	振幅	\
0	2020-01-02	601328	5.66	5.68	5.71	5.66	1204575	685020624.0	0.89	
1	2020-01-03	601328	5.70	5.69	5.70	5.68	792264	450865568.0	0.35	

2	2020-01-06	601328	5.68	5.66	5.70	5.65	1102799	626513152.0	0.88
3	2020-01-07	601328	5.67	5.69	5.72	5.66	1009756	574954032.0	1.06
4	2020-01-08	601328	5.67	5.64	5.67	5.63	884545	499286960.0	0.70

	涨跌幅	涨跌额	换手率
0	0.89	0.05	0.31
1	0.18	0.01	0.20
2	-0.53	-0.03	0.28
3	0.53	0.03	0.26
4	-0.88	-0.05	0.23

清洗股票数据

1. 只保留日期 和 收盘价

```
price_clean = hist_df[["日期", "收盘"]].copy()
```

2. 日期转为 datetime

```
price_clean["日期"] = pd.to_datetime(price_clean["日期"])
```

3. 按日期排序

```
price_clean = price_clean.sort_values("日期")
```

4. 设置日期为索引

```
price_clean = price_clean.set_index("日期")
```

5. 重命名字段

```
price_clean = price_clean.rename(columns={"收盘": "close"})
```

6. 基础检查

```
print(price_clean.head())
```

```
print(price_clean.tail())
```

```
print(price_clean.index.is_monotonic_increasing)
```

```
print(f"价格序列长度: {len(price_clean)}")
```

```
close
日期
2020-01-02    5.68
2020-01-03    5.69
2020-01-06    5.66
2020-01-07    5.69
2020-01-08    5.64
close
```

```
日期
2024-12-25    7.74
2024-12-26    7.70
2024-12-27    7.71
2024-12-30    7.82
2024-12-31    7.77
True
价格序列长度：1212
```

三、公司财务绩效分析

```
# 财务数据的报告期有 5 年
report_dates = ["20201231", "20211231", "20221231", "20231231",
"20241231"]
```

导入资产负债表数据

```
balance_list = []
for date in report_dates:
    # 取“全市场”资产负债表
    df_all = ak.stock_zcfz_em(date=date)
    # 筛选交通银行
    df_one = df_all[df_all["股票代码"] == stock_code].copy()
    # 加报告期
    df_one["报告期"] = date
    balance_list.append(df_one)
balance_df = pd.concat(balance_list, ignore_index=True)
print("资产负债表前 5 行：")
print(balance_df.head())
print("\n 资产负债表字段名：")
print(balance_df.columns)
```

资产负债表前 5 行：

	序号	股票代码	股票简称	资产-货币资金	资产-应收账款	资产-存货
	资产-总资产	资产-总资产同比	\			
0	4788	601328	交通银行	NaN	NaN	NaN
1	4326	601328	交通银行	NaN	NaN	NaN
2	4744	601328	交通银行	NaN	NaN	NaN
3	4958	601328	交通银行	NaN	NaN	NaN

4	492	601328	交通银行	NaN	NaN	NaN	1.490072e+13	5.975937
---	-----	--------	------	-----	-----	-----	--------------	----------

	负债-应付账款 负债率	负债-预收账款 股东权益合计 \	负债-总负债	负债-总负债同比	资产
0	NaN	NaN	9.818988e+12	7.845409	91.786693
1	NaN	NaN	1.068852e+13	8.855627	91.623038
2	NaN	NaN	1.195805e+13	11.864672	92.044673
3	NaN	NaN	1.296102e+13	8.387430	92.180561
4	NaN	NaN	1.374512e+13	6.049662	92.244689

	公告日期	报告期
0	2022-03-26	20201231
1	2023-03-31	20211231
2	2024-03-28	20221231
3	2025-03-22	20231231
4	2025-10-31	20241231

资产负债表字段名:

```
Index(['序号', '股票代码', '股票简称', '资产-货币资金', '资产-应收账款', '资产-存货', '资产-总资产',
      '资产-总资产同比', '负债-应付账款', '负债-预收账款', '负债-总负债', '负债-总负债同比', '资产负债率',
      '股东权益合计', '公告日期', '报告期'],
      dtype='object')
```

保存资产负债表 (5 年)

```
balance_df.to_csv("annual_reports/balance_5y.csv", index=False, encoding="utf-8-sig")
print("资产负债表已保存: annual_reports/balance_5y.csv")
```

导入利润表

```
income_list = []
for date in report_dates:
    df_all = ak.stock_lrb_em(date=date) # 全市场利润表 (该报告期)
    df_all["股票代码"] = df_all["股票代码"].astype(str)
    df_one = df_all[df_all["股票代码"] == stock_code].copy()
    df_one["报告期"] = date
    income_list.append(df_one)
income_df = pd.concat(income_list, ignore_index=True)
```

```

print("利润表前 5 行：")
print(income_df.head())
print("\n 利润表字段名：")
print(income_df.columns)
income_df.to_csv("annual_reports/income_lrb_5y.csv",
index=False, encoding="utf-8-sig")
print("\n 已保存：annual_reports/income_lrb_5y.csv")

```

利润表前 5 行：

	序号	股票代码	股票简称	净利润	净利润同比	营业总收入
			营业总收入同比 \			
0	4791	601328	交通银行	7.827400e+10	1.28	2.462000e+11 5.905227
1	4323	601328	交通银行	8.758100e+10	11.89	2.693900e+11 9.419171
2	4742	601328	交通银行	9.210200e+10	5.22	2.567960e+11 1.331898
3	4957	601328	交通银行	9.272800e+10	0.68	2.575950e+11 0.311142
4	4957	601328	交通银行	9.358600e+10	0.93	2.598260e+11 0.866088

	营业总支出-营业支出	营业总支出-销售费用	营业总支出-管理费用	营业总支出-财务费用	营业总支出-营业总支出 \
0	NaN	NaN	6.600400e+10	NaN	1.599870e+11
1	NaN	NaN	7.454500e+10	NaN	1.756370e+11
2	NaN	NaN	7.615100e+10	NaN	1.589290e+11
3	NaN	NaN	7.736900e+10	NaN	1.581640e+11
4	NaN	NaN	7.768700e+10	NaN	1.564750e+11

	营业利润	利润总额	公告日期	报告期
0	8.621300e+10	8.642500e+10	2022-03-26	20201231
1	9.375300e+10	9.395900e+10	2023-03-31	20211231
2	9.786700e+10	9.811500e+10	2024-03-28	20221231
3	9.943100e+10	9.969800e+10	2025-03-22	20231231
4	1.033510e+11	1.034750e+11	2025-03-22	20241231

利润表字段名：

```

Index(['序号', '股票代码', '股票简称', '净利润', '净利润同比', '营业总收入',
'营业总收入同比', '营业总支出-营业支出',
'营业总支出-销售费用', '营业总支出-管理费用', '营业总支出-财务费用',
'营业总支出-营业总支出', '营业利润', '利润总额',
'公告日期', '报告期'],
dtype='object')

```

已保存：annual_reports/income_lrb_5y.csv

```

# 现金流量表
cashflow_list = []
for date in report_dates:
    df_all = ak.stock_xjll_em(date=date) # 全市场现金流量
    表（该报告期）
    df_all["股票代码"] = df_all["股票代码"].astype(str)

    df_one = df_all[df_all["股票代码"] == stock_code].copy()
    df_one["报告期"] = date
    cashflow_list.append(df_one)
cashflow_df = pd.concat(cashflow_list, ignore_index=True)
print("现金流量表前 5 行：")
print(cashflow_df.head())
print("\n 现金流量表字段名：")
print(cashflow_df.columns)
cashflow_df.to_csv("annual_reports/cashflow_xjll_5y.csv",
index=False, encoding="utf-8-sig")
print("\n 已保存：annual_reports/cashflow_xjll_5y.csv")

```

现金流量表前 5 行：

序号	股票代码	股票简称	净现金流-净现金流	净现金流-同比增长	经营性现金流-现金流量净额	经营性现金流-净现金流占比 \
0	4793	601328 交通银行	1.393850e+11	283.989598	1.493980e+11	107.183700
1	4325	601328 交通银行	-1.128120e+11	-180.935538	-3.477500e+10	-30.825621
2	4744	601328 交通银行	5.449500e+10	148.306031	3.682210e+11	675.696853
3	4956	601328 交通银行	2.665800e+10	-51.081751	1.373230e+11	515.128667
4	4956	601328 交通银行	-1.135110e+11	-525.804637	-5.734800e+10	-50.521976

投资性现金流-现金流量净额	投资性现金流-净现金流占比	融资性现金流-现金流量净额	融资性现金流-净现金流占比	公告日期 \
0	-9.459100e+10	-67.863113	8.827700e+10	63.333214 2022-03-26
1	-7.554800e+10	-66.968053	1.306000e+09	1.157678 2023-03-31
2	-2.848970e+11	-522.794752	-3.297500e+10	-60.510139 2024-03-28
3	-1.166230e+11	-437.478430	4.888000e+09	18.335959 2025-03-22
4	-9.808900e+10	-86.413652	4.252000e+10	37.458925 2025-03-22

报告期

```

0 20201231
1 20211231

```

现金流量表字段名:

四、宏观数据获取

10

```

# 生成日期列 + 同时保留 GDP 绝对值（累计量）与同比
gdp_a = gdp.copy()
# 1) 先把“季度”复制一份到 date（字符串）
gdp_a["date"] = gdp_a["季度"].astype(str)
# 2) 把不同季度写法，统一替换为季度末日期（最直观、最好讲）
gdp_a["date"] = gdp_a["date"].str.replace("年第1季度",
"-03-31", regex=False)
gdp_a["date"] = gdp_a["date"].str.replace("年第1-2季度",
"-06-30", regex=False)
gdp_a["date"] = gdp_a["date"].str.replace("年第1-3季度",
"-09-30", regex=False)
gdp_a["date"] = gdp_a["date"].str.replace("年第1-4季度",
"-12-31", regex=False)
# 3) 转为日期类型
gdp_a["date"] = pd.to_datetime(gdp_a["date"])
# 4) 同时保留 GDP 绝对值 + 同比（并改列名更清晰）
gdp_a = gdp_a.rename(columns={"国内生产总值-绝对值":
"GDP","国内生产总值-同比增长": "GDPrate"})
gdp_a=gdp_a[["date","GDP","GDPrate"]].sort_values("date").reset
_index(drop=True)

# 5) 保存
gdp_a.to_csv("annual_reports/macro_gdp_2020_2024.csv",
index=False,
encoding="utf-8-sig")
print(gdp_a.head())
print(gdp_a.tail())

```

	date	GDP	GDPrate
0	2006-03-31	47831.7	12.4
1	2006-06-30	101247.0	13.1
2	2006-09-30	158067.6	12.7
3	2006-12-31	222578.4	12.7
4	2007-03-31	58080.6	13.6

	date	GDP	GDPrate
74	2024-09-30	975357.4	4.8
75	2024-12-31	1349083.5	5.0
76	2025-03-31	318758.0	5.4
77	2025-06-30	660535.8	5.3
78	2025-09-30	1015036.1	5.2

下载 cpi 数据

```

cpi = ak.macro_china_cpi()
print(cpi.columns)
print(cpi.head())

```

```

Index(['月份', '全国-当月', '全国-同比增长', '全国-环比增长', '全国-累计', '城市-当月', '城市-同比增长',
      '城市-环比增长', '城市-累计', '农村-当月', '农村-同比增长', '农村-环比增长', '农村-累计'],
      dtype='object')

```

	月份	全国-当月	全国-同比增长	全国-环比增长	全国-累计	城市-当月	城市-同比增长	城市-环比增长	城市-累计
0	2025 年 11 月份	100.7	0.7	-0.1	100.0	100.7	0.7	-0.1	100.0
1	2025 年 10 月份	100.2	0.2	0.2	99.9	100.3	0.3	0.2	100.0
2	2025 年 09 月份	99.7	-0.3	0.1	99.9	99.8	-0.2	0.0	99.9
3	2025 年 08 月份	99.6	-0.4	0.0	99.9	99.7	-0.3	0.0	100.0
4	2025 年 07 月份	100.0	0.0	0.4	99.9	100.0	0.0	0.4	100.0

	农村-当月	农村-同比增长	农村-环比增长	农村-累计
0	100.4	0.4	0.0	99.7
1	99.8	-0.2	0.1	99.7
2	99.5	-0.5	0.2	99.7
3	99.4	-0.6	0.1	99.7
4	99.7	-0.3	0.3	99.7

进行数据处理

1) 处理“月份”（2025 年 11 月份 → 2025-11-01）

```

cpi["date"] = (cpi["月份"]
               .astype(str)
               .str.replace("年", "-", regex=False)
               .str.replace("月份", "-01", regex=False))
cpi["date"] = pd.to_datetime(cpi["date"])

```

```

# 2) 只保留最常用的 CPI 指标 (全国同比)
cpi_simple = cpi.rename(columns={"全国-同比增长": "CPIrate"})
cpi_simple = cpi_simple[["date", "CPIrate"]].sort_values("date")
# 3) 筛选最近 5 年 (2020 - 2024)
cpi_5y = cpi_simple[(cpi_simple["date"] >= "2020-01-01") &
                    (cpi_simple["date"] <= "2024-12-31")]
# 4) 保存
cpi_5y.to_csv("annual_reports/macro_cpi_2020_2024.csv",
              index=False,
              encoding="utf-8-sig")
print("CPI (全国同比, 2020 - 2024) 已保存")
print(cpi_5y.head())

```

进行数据处理

```

# 1) 处理 “月份” (2025 年 11 月份 → 2025-11-01)
cpi["date"] = (cpi["月份"]
               .astype(str)
               .str.replace("年", "-", regex=False)
               .str.replace("月份", "-01", regex=False))
cpi["date"] = pd.to_datetime(cpi["date"])
# 2) 只保留最常用的 CPI 指标 (全国同比)
cpi_simple = cpi.rename(columns={"全国-同比增长": "CPIrate"})
cpi_simple = cpi_simple[["date", "CPIrate"]].sort_values("date")

```

```

# 3) 筛选最近 5 年 (2020 - 2024)
cpi_5y = cpi_simple[(cpi_simple["date"] >= "2020-01-01") &
                    (cpi_simple["date"] <= "2024-12-31")]
# 4) 保存
cpi_5y.to_csv("annual_reports/macro_cpi_2020_2024.csv",
              index=False, encoding="utf-8-sig")
print("CPI (全国同比, 2020 - 2024) 已保存")
print(cpi_5y.head())

```

计算对数收益率

```

# 复制一份数据，避免影响原始 price_clean
final_df = price_clean.copy().sort_index()
# 1. 计算对数收益率
final_df["log_ret"] = np.log(final_df["close"]).diff()
# 看一眼结果
final_df.head()

```

```

      关闭 log_ret
日期
2020-01-02 5.68   NaN
2020-01-03 5.69   0.001759
2020-01-06 5.66  -0.005286
2020-01-07 5.69   0.005286
2020-01-08 5.64  -0.008826

```

计算波动率

```

# 2. 计算 20 日滚动年化波动率
window = 20          # 20 个交易日
trading_days = 252    # 一年交易日数
# 20 日滚动年化波动率
window = 20
final_df["volatility"] = (final_df["log_ret"]
    .rolling(window)
    .std()
    * np.sqrt(trading_days))
final_df.head(25)

```

```

      关闭 log_ret 波动率
日期
2020-01-02 5.68   NaNNaN
2020-01-03 5.69   0.001759 NaN
2020-01-06 5.66  -0.005286NaN
2020-01-07 5.69   0.005286 NaN
2020-01-08 5.64  -0.008826NaN

```

2020-01-09	5.65	0.001771	NaN
2020-01-10	5.64	-0.001771	NaN
2020-01-13	5.65	0.001771	NaN
2020-01-14	5.67	0.003534	NaN
2020-01-15	5.62	-0.008857	NaN
2020-01-16	5.61	-0.001781	NaN
2020-01-17	5.61	0.000000	NaN
2020-01-20	5.62	0.001781	NaN
2020-01-21	5.57	-0.008937	NaN
2020-01-22	5.54	-0.005401	NaN
2020-01-23	5.47	-0.012716	NaN
2020-02-03	5.11	-0.068079	NaN
2020-02-04	5.20	0.017459	NaN
2020-02-05	5.20	0.000000	NaN
2020-02-06	5.24	0.007663	NaN
2020-02-07	5.25	0.001907	0.262780
2020-02-10	5.23	-0.003817	0.261921
2020-02-11	5.26	0.005720	0.264228
2020-02-12	5.28	0.003795	0.263610
2020-02-13	5.24	-0.007605	0.263333

可视化，先找出高波动的阶段

高波动标记（80%分位）

threshold = final_df["volatility"].quantile(0.8)

标记是否处于高波动阶段

final_df["high_vol"] = final_df["volatility"] > threshold

看一眼结果

final_df.head(30)

	关闭	log_ret	波动率	high_vol
日期				
2020-01-02	5.68	NaN	NaN	错误
2020-01-03	5.69	0.001759	NaN	错误
2020-01-06	5.66	-0.005286	NaN	错误

2020-01-07	5.69	0.005286	NaN	错误
2020-01-08	5.64	-0.008826	NaN	错误
2020-01-09	5.65	0.001771	NaN	错误
2020-01-10	5.64	-0.001771	NaN	错误
2020-01-13	5.65	0.001771	NaN	错误
2020-01-14	5.67	0.003534	NaN	错误
2020-01-15	5.62	-0.008857	NaN	错误
2020-01-16	5.61	-0.001781	NaN	错误
2020-01-17	5.61	0.000000	NaN	错误
2020-01-20	5.62	0.001781	NaN	错误
2020-01-21	5.57	-0.008937	NaN	错误
2020-01-22	5.54	-0.005401	NaN	错误
2020-01-23	5.47	-0.012716	NaN	错误
2020-02-03	5.11	-0.068079	NaN	错误
2020-02-04	5.20	0.017459	NaN	错误
2020-02-05	5.20	0.000000	NaN	错误
2020-02-06	5.24	0.007663	NaN	错误
2020-02-07	5.25	0.001907	0.262780	确实如此
2020-02-10	5.23	-0.003817	0.261921	确实如此
2020-02-11	5.26	0.005720	0.264228	确实如此
2020-02-12	5.28	0.003795	0.263610	确实如此
2020-02-13	5.24	-0.007605	0.263333	确实如此
2020-02-14	5.28	0.007605	0.265737	确实如此
2020-02-17	5.34	0.011300	0.270793	确实如此
2020-02-18	5.31	-0.005634	0.270435	确实如此
2020-02-19	5.32	0.001881	0.269960	确实如此
2020-02-20	5.35	0.005623	0.270817	确实如此

```

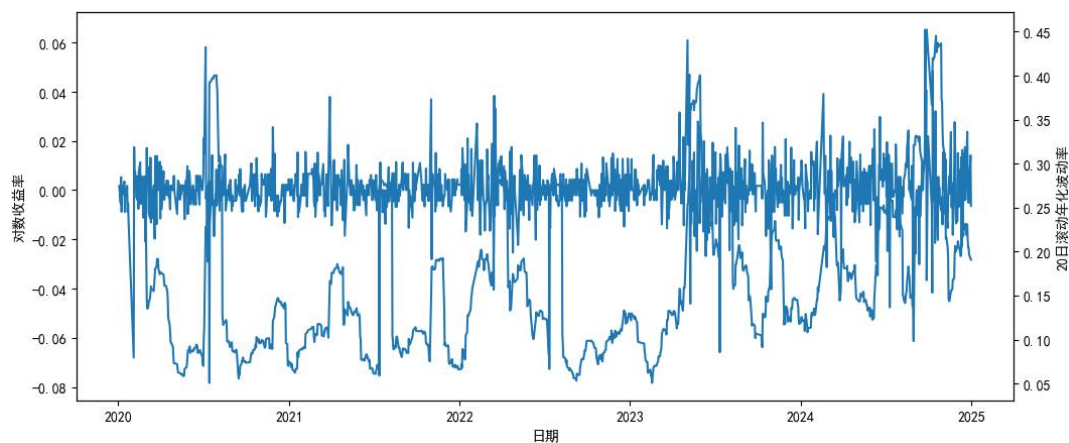
# 画图（收益率+波动率+高亮阶段）
df = final_df.copy()
fig, ax1 = plt.subplots(figsize=(12, 5))
# 左轴：收益率
ax1.plot(df.index, df["log_ret"])
ax1.set_xlabel("日期")

```

```

ax1.set_ylabel("对数收益率")
# 右轴：波动率
ax2 = ax1.twinx()
ax2.plot(df.index, df["volatility"])
ax2.set_ylabel("20 日滚动年化波动率")
# 高亮：高波动阶段（背景）
ax1.fill_between(df.index, 0, 1,
                 where=df["high_vol"],
                 transform=ax1.get_xaxis_transform(),
                 alpha=0.2)
ax1.set_title("收益率与波动率（高波动阶段高亮）")
plt.tight_layout()
plt.show()

```



```

# 把图中线条颜色换种效果
df = final_df.copy()
fig, ax1 = plt.subplots(figsize=(12, 5))
# =====
# 左轴：收益率（蓝色）
# =====
ax1.plot(
    df.index,
    df["log_ret"],
    label="对数收益率",
    linewidth=1,

```

```

        color="tab:blue")
ax1.set_xlabel("日期")
ax1.set_ylabel("对数收益率", color="tab:blue")
ax1.tick_params(axis="y", labelcolor="tab:blue")

# =====
# 右轴：波动率（红色）
# =====
ax2 = ax1.twinx()
ax2.plot(
    df.index,
    df["volatility"],
    label="20 日滚动年化波动率",
    linewidth=1.5,
    color="tab:red")
ax2.set_ylabel("20 日滚动年化波动率", color="tab:red")
ax2.tick_params(axis="y", labelcolor="tab:red")

# =====
# 高波动阶段：深灰色背景
# =====
ax1.fill_between(
    df.index,
    0, 1,
    where=df["high_vol"],
    transform=ax1.get_xaxis_transform(),
    color="yellow",
    alpha=0.35,
    label="高波动阶段")

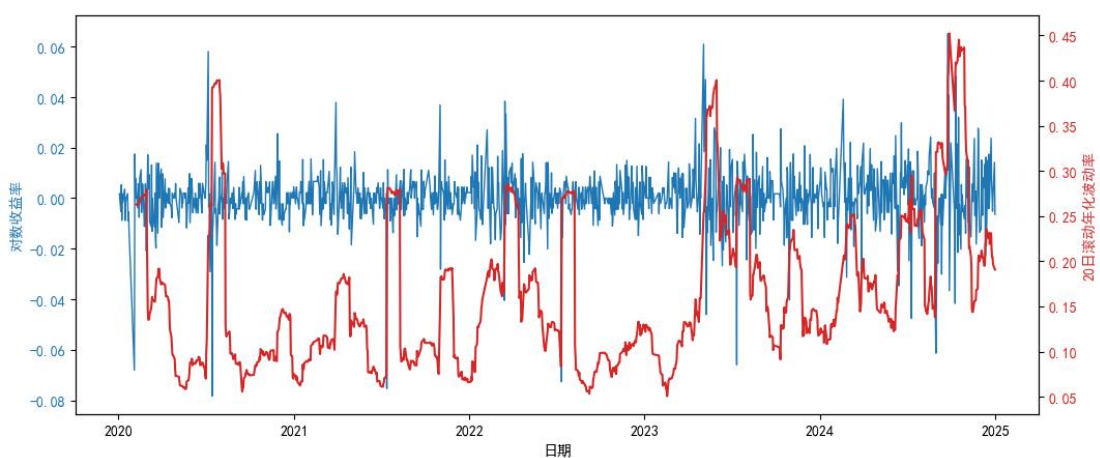
# =====
# 标题 & 图例
# =====
ax1.set_title("收益率与波动率（高波动阶段高亮）")

```

```

# 合并双轴图例
lines_1, labels_1 = ax1.get_legend_handles_labels()
lines_2, labels_2 = ax2.get_legend_handles_labels()
ax1.legend(
    lines_1 + lines_2,
    labels_1 + labels_2,
    loc="upper left")
plt.tight_layout()
plt.show()

```



年度对比：把三表的关键指标按年汇总到一张“年度财务指标表”

高波动年份对照：找出某个（或多个）高波动年份，看财务指标是否“同步恶化”

```

# =====
# 1) 读入三张表（你上传的文件名）
# =====
balance = pd.read_csv("annual_reports/balance_5y.csv")
income = pd.read_csv("annual_reports/income_lrb_5y.csv")
cashflow = pd.read_csv("annual_reports/cashflow_xjll_5y.csv")
# =====
# 2) 从“报告期”提取年份（20201231 -> 2020）
# =====
def add_year(df):
    df = df.copy()

```

```

df["year"] = df["报告期"].astype(str).str[:4].astype(int)
return df
balance = add_year(balance)
income = add_year(income)
cashflow = add_year(cashflow)
# =====
# 3) 选取关键指标（你这三表的列名就是这些）
# =====
bal = balance[["year", "资产-总资产", "负债-总负债", "资产负债率", "股东权益合计"]].copy()
inc = income[["year", "营业总收入", "净利润", "营业利润", "利润总额"]].copy()
cf = cashflow[["year", "经营性现金流-现金流量净额", "投资性现金流-现金流量净额", "融资性现金流-现金流量净额", "净现金流-净现金流"]].copy()
# =====
# 4) 合并成“年度财务指标表”
# =====
fin = bal.merge(inc, on="year", how="inner").merge(cf, on="year", how="inner")
fin = fin.sort_values("year").reset_index(drop=True)
# =====
# 5) 增加几个常用派生指标（简单但很有用）
# =====
fin["净利率"] = fin["净利润"] / fin["营业总收入"]
fin["经营现金流/净利润"] = fin["经营性现金流-现金流量净额"] / fin["净利润"]
# =====
# 6) 计算同比变化（和上一年相比）
# =====
cols_to_yoy = [ "营业总收入", "净利润", "资产-总资产", "负债-总负债", "经营性现金流-现金流量净额", "投资性现金流-现金流量净额", "净利率", "资产负债率"]
for c in cols_to_yoy:

```

```

fin[c + "_YoY"] = fin[c].pct_change()
print("年度财务指标表（核心列）预览：")
show_cols = ["year",
              "营业总收入","净利润","净利率",
              "资产-总资产","负债-总负债","资产负债率",
              "经营性现金流-现金流量净额","投资性现金流-
现金流量净额", "经营现金流/净利润"]
print(fin[show_cols])

```

年度财务指标表（核心列）预览：

	year	营业总收入	净利润	净利率	资产-总资产	负债-总
0	2020	2.462000e+11	7.827400e+10	0.317929	1.069762e+13	9.818988e+12
1	2021	2.693900e+11	8.758100e+10	0.325109	1.166576e+13	1.068852e+13
2	2022	2.567960e+11	9.210200e+10	0.358658	1.299157e+13	1.195805e+13
3	2023	2.575950e+11	9.272800e+10	0.359976	1.406047e+13	1.296102e+13
4	2024	2.598260e+11	9.358600e+10	0.360187	1.490072e+13	1.374512e+13

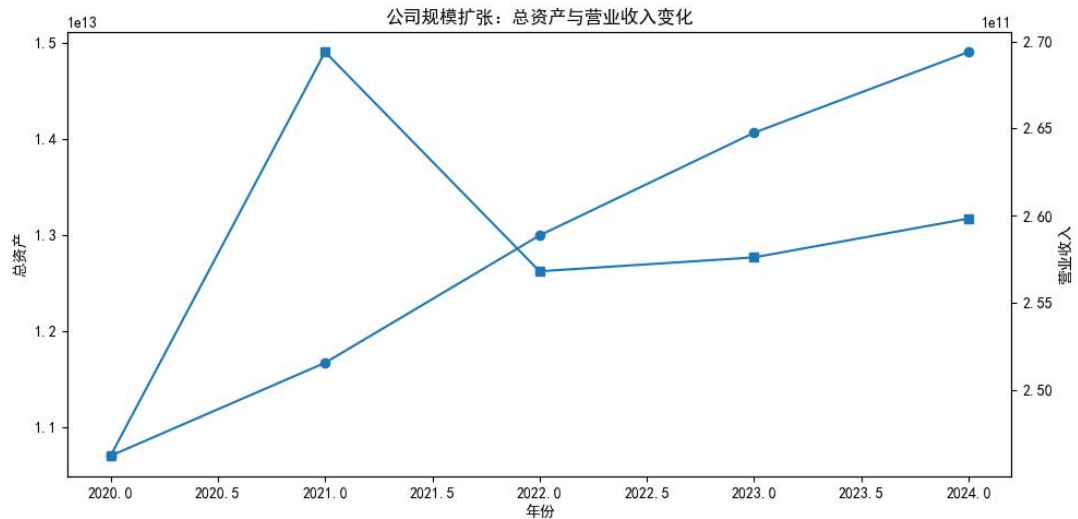
	资产负债率	经营性现金流-现金流量净额	投资性现金流-现金流量净额	经营现金流/净利润
0	91.786693	1.493980e+11	-9.459100e+10	1.908654
1	91.623038	-3.477500e+10	-7.554800e+10	-0.397061
2	92.044673	3.682210e+11	-2.848970e+11	3.997970
3	92.180561	1.373230e+11	-1.166230e+11	1.480923
4	92.244689	-5.734800e+10	-9.808900e+10	-0.612784

```

# 可视化
# 总资产 vs 营业收入
df = fin.copy()
fig, ax1 = plt.subplots(figsize=(10, 5))
# 左轴：总资产
ax1.plot(df["year"], df["资产-总资产"], marker="o")
ax1.set_xlabel("年份")
ax1.set_ylabel("总资产")
# 右轴：营业收入
ax2 = ax1.twinx()
ax2.plot(df["year"], df["营业总收入"], marker="s")

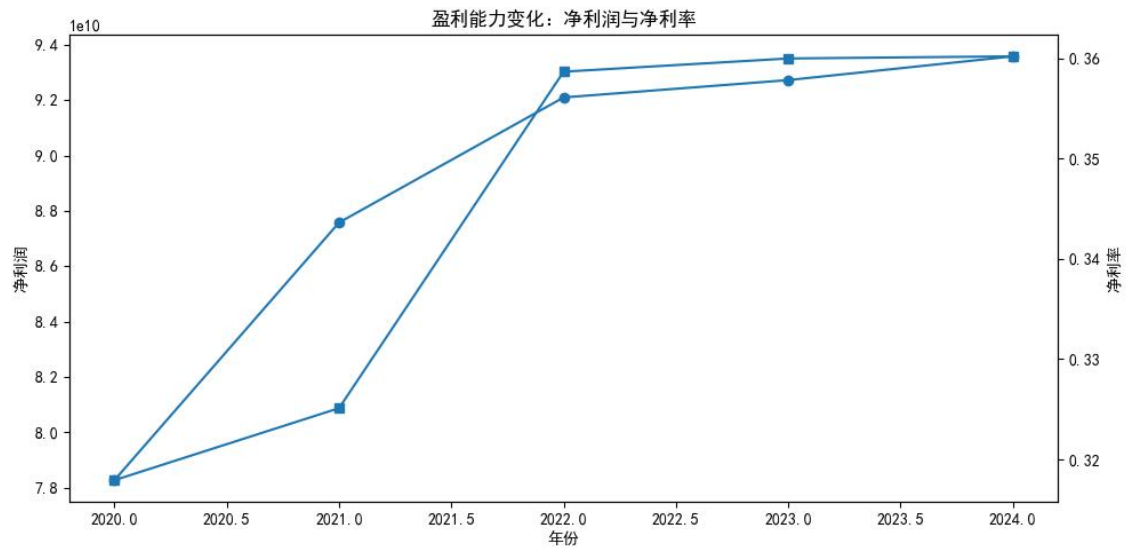
```

```
ax2.set_ylabel("营业收入")
ax1.set_title("公司规模扩张：总资产与营业收入变化")
plt.tight_layout()
plt.show()
```



解读： 公司总资产持续扩张，显示出明显的资本开支与扩产特征；营业收入随资产扩张同步变化，表明扩张并非脱离主营业务。

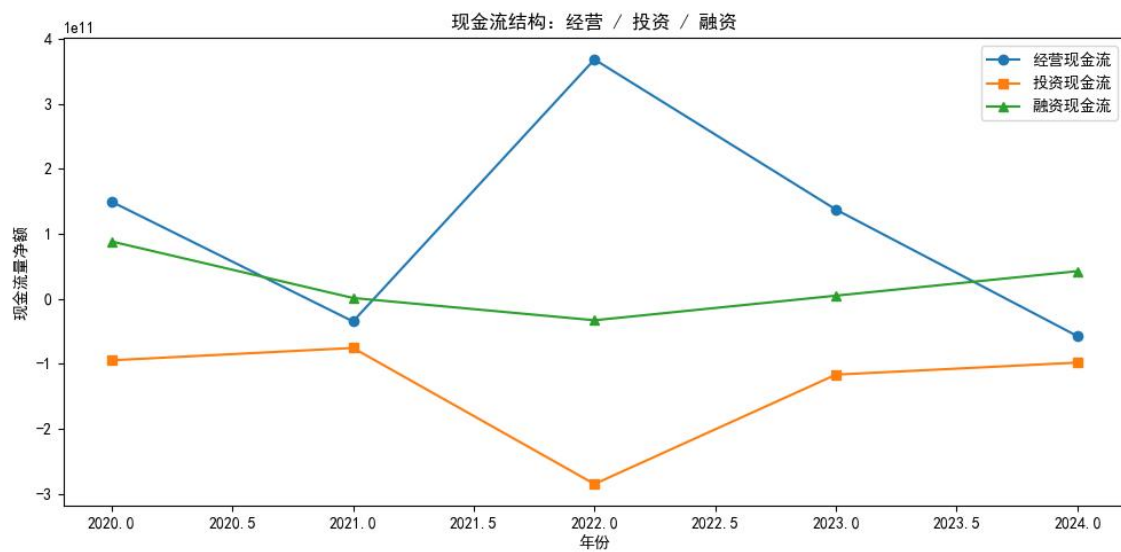
```
# 盈利能力是否“跟得上扩张”（利润表核心）
fig, ax1 = plt.subplots(figsize=(10, 5))
# 左轴：净利润
ax1.plot(df["year"], df["净利润"], marker="o")
ax1.set_xlabel("年份")
ax1.set_ylabel("净利润")
# 右轴：净利率
ax2 = ax1.twinx()
ax2.plot(df["year"], df["净利率"], marker="s")
ax2.set_ylabel("净利率")
ax1.set_title("盈利能力变化：净利润与净利率")
plt.tight_layout()
plt.show()
```



解读：尽管收入规模保持，但净利率波动较大，显示成本结构与行业周期对盈利形成明显制约；

当前阶段盈利释放尚未与资产扩张完全匹配。

```
# 现金流是否支持战略扩张（现金流量表核心）
plt.figure(figsize=(10, 5))
plt.plot(df["year"], df["经营性现金流-现金流量净额"],
marker="o")
plt.plot(df["year"], df["投资性现金流-现金流量净额"],
marker="s")
plt.plot(df["year"], df["融资性现金流-现金流量净额"],
marker="^")
plt.xlabel("年份")
plt.ylabel("现金流量净额")
plt.title("现金流结构：经营 / 投资 / 融资")
plt.legend(["经营现金流", "投资现金流", "融资现金流"])
plt.tight_layout()
plt.show()
```



解读：投资现金流长期为负，反映持续的资本性支出；经营现金流波动较大，尚不足以完全覆盖投资支出；融资现金流在部分年份起到对冲作用，表明扩张对外部资金与政策环境存在一定依赖。

宏观数据 → 年度指标

```
gdp_year = gdp_a.copy()
gdp_year["year"] = gdp_year["date"].dt.year
# 每年只保留最后一个季度（累计口径）
gdp_year = (gdp_year
            .sort_values("date")
            .groupby("year")
            .tail(1)
            .reset_index(drop=True))
```

只保留最近 5 年的数据

```
gdp_5y = gdp_year[(gdp_year["year"] >= 2020) &
                  (gdp_year["year"] <= 2024)].copy()
print(gdp_5y)
```

	date	GDP	GDPRate	year
14	2020-12-31	1034867.6	2.3	2020
15	2021-12-31	1173823.0	8.6	2021

16	2022-12-31	1234029.4	3.1	2022
17	2023-12-31	1294271.7	5.4	2023
18	2024-12-31	1349083.5	5.0	2024

```
# CPI (月度 → 年度)
cpi_year = cpi_5y.copy()
cpi_year["year"] = cpi_year["date"].dt.year
cpi_year = (cpi_year
            .groupby("year")["CPIrate"]
            .mean()
            .reset_index())
print(cpi_year)
```

	year	CPIrate
0	2020	2.508333
1	2021	0.916667
2	2022	1.958333
3	2023	0.241667
4	2024	0.241667

```
# 生成最终“公司研究主表”
# 以财务数据为核心（公司研究主线）
final_panel = fin.copy()
# 合并股票年度指标
final_panel = final_panel.merge(stock_year, on="year",
how="left")
# 合并 GDP
final_panel = final_panel.merge(gdp_year, on="year",
how="left")
# 合并 CPI
final_panel = final_panel.merge(cpi_year, on="year",
how="left")
print("最终对齐后的分析主表：")
print(final_panel)
```

最终对齐后的分析主表：

	year	资产-总资产		负债-总负债		资产负债率		股东权益合计	
营业总收入 \									
0	2020	1.069762e+13	9.818988e+12	91.786693	8.786280e+11	2.462000e+11			
1	2021	1.166576e+13	1.068852e+13	91.623038	9.772360e+11	2.693900e+11			
2	2022	1.299157e+13	1.195805e+13	92.044673	1.033522e+12	2.567960e+11			
3	2023	1.406047e+13	1.296102e+13	92.180561	1.099450e+12	2.575950e+11			
4	2024	1.490072e+13	1.374512e+13	92.244689	1.155597e+12	2.598260e+11			
		净利润		营业利润		利润总额		经营性现金流-现金流量净额 ... \	
0	7.827400e+10	8.621300e+10	8.642500e+10	1.493980e+11	...				
1	8.758100e+10	9.375300e+10	9.395900e+10	-3.477500e+10	...				
2	9.210200e+10	9.786700e+10	9.811500e+10	3.682210e+11	...				
3	9.272800e+10	9.943100e+10	9.969800e+10	1.373230e+11	...				
4	9.358600e+10	1.033510e+11	1.034750e+11	-5.734800e+10	...				
		投资性现金流-现金流量净额_YoY		净利率_YoY		资产负债率_YoY		avg_ret	avg_vol high_v
ol_ratio \									
0		NaN	NaN	NaN	-0.000981	0.145623		0.160494	
1		-0.201319	0.022584	-0.001783	0.000118	0.123999		0.082305	
2		2.771073	0.103195	0.004602	0.000115	0.145381		0.161157	
3		-0.590649	0.003674	0.001476	0.000791	0.178132		0.214876	
4		-0.158922	0.000587	0.000696	0.001251	0.218828		0.367769	
	date	GDP	GDPrate	CPIrate					
0	2020-12-31	1034867.6	2.3	2.508333					
1	2021-12-31	1173823.0	8.6	0.916667					
2	2022-12-31	1234029.4	3.1	1.958333					
3	2023-12-31	1294271.7	5.4	0.241667					
4	2024-12-31	1349083.5	5.0	0.241667					

[5 rows x 30 columns]

```
# 一张图讲清“宏观 → 市场 → 公司”的传导
df = final_panel.copy().sort_values("year")
# =====
# 1) 选三个指标（宏观 / 市场 / 公司）
# 你只要保证这些列在 df 里存在即可
# =====
# 宏观：优先用 GDPrate（你现在就是这个列名）
```

```

macro_col = "GDPPrate" if "GDPPrate" in df.columns else ("GDP_yoy"
if "GDP_yoy" in df.columns else None)
# 市场：优先用 avg_vol（年均波动率），没有就用
high_vol_ratio（高波动天数占比）
market_col = "avg_vol" if "avg_vol" in df.columns else
("high_vol_ratio" if "high_vol_ratio" in df.columns else None)
# 公司：优先用 净利润同比；没有就用 营收同比；再没有就用
净利率同比
company_col = None
for c in ["净利润_YoY", "营业总收入_YoY", "净利率_YoY"]:
    if c in df.columns:
        company_col = c
        break
print("宏观指标：", macro_col)
print("市场指标：", market_col)
print("公司指标：", company_col)

# 检查是否都选到了
if (macro_col is None) or (market_col is None) or (company_col
is None):raise ValueError("你的 final_panel 里缺少必要列。请把
df.columns 打印出来，我帮你对齐列名。")
# =====
# 2) 把三个序列都转成“指数=100”（同一尺度更适合一张图
讲故事）
# 解释给学生：把每个指标的起点当作 100，看相对变化
# =====
def to_index100(s):
    s = s.astype(float)
    first = s.dropna().iloc[0]
    return (s / first) * 100
plot_df = df[["year", macro_col, market_col,
company_col]].copy()
plot_df["宏观指数"] = to_index100(plot_df[macro_col])
plot_df["市场指数"] = to_index100(plot_df[market_col])

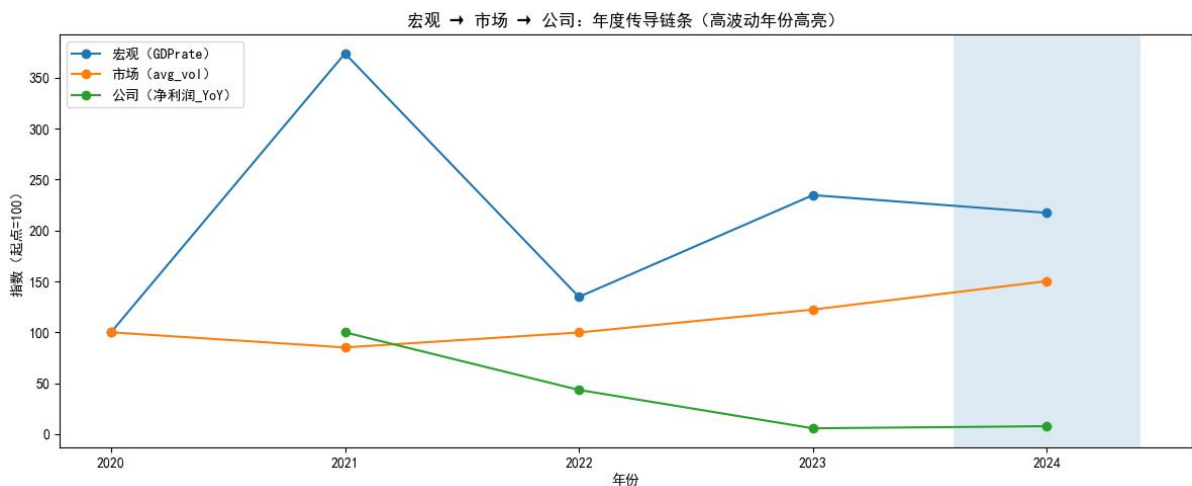
```

```

plot_df["公司指数"] = to_index100(plot_df[company_col])
# =====
# 3) 高波动年份：如果你有 high_vol_ratio，就用它自动找“最
波动的一年”
# 否则就不高亮
# =====
high_years = []
if "high_vol_ratio" in df.columns:
    high_year = int(df.sort_values("high_vol_ratio",
ascending=False).iloc[0]["year"])
    high_years = [high_year]
    print("识别出的高波动年份：", high_years)
# =====
# 4) 画一张图：宏观→市场→公司
# =====
plt.figure(figsize=(12, 5))
plt.plot(plot_df["year"], plot_df["宏观指数"], marker="o")
plt.plot(plot_df["year"], plot_df["市场指数"], marker="o")
plt.plot(plot_df["year"], plot_df["公司指数"], marker="o")

# 高波动年份背景高亮
for y in high_years:
    plt.axvspan(y - 0.4, y + 0.4, alpha=0.15)
plt.xlabel("年份")
plt.ylabel("指数（起点=100）")
plt.title("宏观 → 市场 → 公司：年度传导链条（高波动年份
高亮）")
plt.legend([f"宏观（{macro_col}）", f"市场（{market_col}）", f"
公司（{company_col}）"])
plt.tight_layout()
plt.show()

```



```
# 识别并读取年报 PDF 文件列表
import pdfplumber
report_dir = "annual_reports"
pdf_files = sorted([os.path.join(report_dir, f)
                     for f in os.listdir(report_dir)
                     if f.lower().endswith(".pdf")])
print("识别到的年报文件：")
for f in pdf_files:
    print(f)
```

字符数： 334972

前 300 字预览：

公司代码：601328 公司简称：交通银行 交通银行股份有限公司 2020 年度报告

目 录 重要提示 3 释义 4 公司基本情况 5 财务摘要 9 董事长致辞 12 行长致辞 14 经营情况讨论与分析 16 一、经济金融形势 16 二、财务报表分析 16 三、业务回顾 30 四、风险管理 42 五、深化改革 49 六、展望 50 股份变动及股东情况 52 董事、监事、高级管理人员和人力资源管理 58 公司治理报告 76 董事会报告 88 监事会报告 93 社会责任 95 重要事项 97 组织架构与机构名录 99 境外银行机构名录 102 董事、监事、高级管理人员对年度报告确认意见 104 备

五、文本分析

```
# PDF → TXT (逐年保存 + 统计报告)
# 1) 输出目录 (没有就创建)
out_dir = "annual_reports/data/txt"
os.makedirs(out_dir, exist_ok=True)
stats = [] # 记录每年的统计信息
for pdf_path in pdf_files:
    file_name = os.path.basename(pdf_path)
    # 2) 从文件名里提取年份 (简单方法: 找连续 4 位数字)
    m = re.search(r"(20\d{2})", file_name)
    year = m.group(1) if m else file_name.replace(".pdf", "")
    texts = []
    total_pages = 0
    empty_pages = 0
    with pdfplumber.open(pdf_path) as pdf:
        total_pages = len(pdf.pages)
        for page in pdf.pages:
            text = page.extract_text()
            if text and text.strip():
                # 简单清洗: 合并空白
                text = re.sub(r"\s+", " ", text)
                texts.append(text)
            else:
                empty_pages += 1
    full_text = "\n".join(texts)
    # 3) 保存为 txt
    txt_path = os.path.join(out_dir, f"{year}.txt")
    with open(txt_path, "w", encoding="utf-8") as f:
        f.write(full_text)
    # 4) 统计信息
    word_count = len(full_text)
    success_rate = (total_pages - empty_pages) / total_pages
if total_pages > 0 else 0
```

```

        empty_ratio = empty_pages / total_pages if
total_pages > 0 else 0
        stats.append([year, total_pages, empty_pages,
round(empty_ratio, 4), round(success_rate, 4), word_count])
        print(f"✔ {year} 年报已保存: {txt_path} | 字符数
={word_count} | 空页比例={empty_ratio:.2%}")
        # 5) 输出汇总报告
        print("\n==== PDF→TXT 汇总报告 =====")
        print("year | 总页数 | 空页数 | 空页比例 | 提取成功率 | 字符
数")
        for row in stats:
            print(row)

# 逐年分词 + 输出 Top30 (按年 + 5 年合并)
txt_dir = "annual_reports/data/txt"
stop_path = "annual_reports/data/stopwords.txt"
# 1) 读取停用词
with open(stop_path, "r", encoding="utf-8") as f:
    stopwords = set([line.strip() for line in f if line.strip()])
# 2) 找到所有 txt
txt_files = sorted([os.path.join(txt_dir, f) for f in
os.listdir(txt_dir) if f.endswith(".txt")])
print("识别到的 TXT: ", [os.path.basename(x) for x in txt_files])
def tokenize(text):
    """中文分词 + 基础清洗 (零基础友好版)"""
    text = re.sub(r"[a-zA-Z0-9]+", " ", text) # 去英数
    text = re.sub(r"[\u4e00-\u9fa5]+", " ", text) # 只保留中
文
    words = jieba.lcut(text)
    # 过滤: 停用词、长度<2
    words = [w for w in words if (len(w) >= 2) and (w not in
stopwords)]
    return words
year_top = {} # 每年 Top30

```

```

all_words = []          # 合并词
for path in txt_files:
    year = os.path.basename(path).replace(".txt", "")
    with open(path, "r", encoding="utf-8") as f:
        text = f.read()
        words = tokenize(text)
        all_words.extend(words)

    top30 = Counter(words).most_common(30)
    year_top[year] = top30
    print(f"\n===== {year} Top30 高频词 =====")
    for w, c in top30:
        print(w, c)

# 合并 Top30
print("\n===== 五年合并 Top30 高频词 =====")
top30_all = Counter(all_words).most_common(30)
for w, c in top30_all:
    print(w, c)

```

===== 2020 Top30 高频词 =====

日止 286
 成本 223
 信用 218
 减值 217
 金融工具 216
 衍生 215
 股东 214
 确认 202
 转移 194
 当期 187
 款项 186
 发行 183
 利率 181

预期	179
现金	177
市场	174
资金	169
优先股	167
事项	166
同业	166
垫款	166
存放	165
工具	165
中国	161
权益	161
分行	160
服务	157
所得税	155
影响	155
存款	154



===== 2021 Top30 高频词 =====

日止 268

成本 224
信用 218
减值 217
股东 216
衍生 213
金融工具 209
转移 203
期内 202
确认 201
发展 188
资金 187
款项 187
当期 187
发行 180
同业 180
风险管理 179
预期 179
利率 175
分行 172
事项 170
现金 169
服务 167
存放 167
垫款 165
市场 164
权益 160
工具 159
企业 158
影响 158



===== 2022 Top30 高频词 =====

信用 217
 日止 214
 股东 211
 确认 202
 发展 202
 期内 202
 服务 196
 成本 195
 衍生 178
 减值 177
 年末 177
 持续 173
 发行 172
 企业 171
 市场 164
 工具 161
 预期 160

成本 203
 期内 197
 发行 190
 减值 188
 中国 186
 持续 184
 年末 180
 企业 178
 衍生 176
 资金 175
 控制 166
 市场 164
 工具 160
 权益 159
 利率 155
 当期 155
 预期 154
 审计 152
 金融工具 151
 信息 148
 发生 148
 内部 146
 绿色 142
 租赁 142

2023 年报词云图



===== 2024 Top30 高频词 =====

信用	217
持续	213
股东	209
服务	209
日止	207
成本	203
发展	201
确认	200
期内	197
发行	197
减值	182
中国	181
控制	166
年末	164
企业	161
合同	159
资金	155
当期	154
工具	153
审计	152
绿色	152
利率	150
衍生	150
预期	149
租赁	144
市场	143
金融工具	141
信息	140
款项	139
权益	138

日止	1191
信用	1093
股东	1053
成本	1048
确认	1030
减值	981
服务	944
期内	944
衍生	932
发行	922
发展	910
金融工具	854
资金	842
当期	842
持续	826
利率	821
预期	821
中国	810
市场	809
工具	798

监事 708



提取 MD&A 计算情感

2020 -> PDF | 字 符 数 : 74476 | 输 出 :
annual_reports\data\mdna_full_3\2020_mdna_full_3.txt

前 200 字预览：经营情况讨论与分析 一、经济金融形势 2020 年，中国坚持稳中求进工作总基调，坚持高质量发展方向不动摇，统筹疫情防控和经济社会发展，国民经济运行总体稳定，脱贫攻坚战取得全面胜利，决胜全面建成小康社会取得决定性成就。全年实现国内生产总值 101.6 万亿元，同比增长 2.3%，成为全球唯一实现经济正增长的主要经济体。“六稳”“六保”成效显著，就业形势好于预

期，国际收支基本平衡，物价水平总体稳定。

2021 -> PDF | 字符数 : 77939 | 输出 :
annual_reports\data\mdna_full_3\2021_mdna_full_3.txt

前 200 字预览：管理层讨论与分析 一、经济金融形势 2021 年，面对内外部诸多风险挑战，在党中央的坚强领导下，中国经济稳中有升，高质量发展取得新成效，构建新发展格局迈出新步伐，实现了“十四五”良好开局。统筹疫情防控和经济社会发展，保供稳价和助企纾困力度加大，粮食产量再创新高，企业经营持续改善，高技术产业较快发展。宏观政策跨周期调节扎实推进，实体经济稳中有升，就业物价总体稳定，进出口增势良好，民生保障坚

2022 -> PDF | 字符数 : 78031 | 输出 :
annual_reports\data\mdna_full_3\2022_mdna_full_3.txt

前 200 字预览：管理层讨论与分析 一、经济金融形势 2022 年，面对复杂严峻的国内外形势和多重超预期因素冲击，在党中央的坚强领导下，我国高效统筹疫情防控和经济社会发展，加快落实稳经济一揽子政策和接续措施，加大保供稳价和助企纾困力度，工业生产总体稳定，经济总量再上新台阶，全年国内生产总值首次超过 120 万亿元人民币。同时，高质量发展不断取得新成效，高技术制造业等新动能快速成长，现代服务业增势较好，粮食产

2023 -> PDF | 字符数 : 61034 | 输出 :
annual_reports\data\mdna_full_3\2023_mdna_full_3.txt

前 200 字预览：管理层讨论与分析 一、经济金融形势 2023 年，全球经济增长动能趋于弱化，各主要经济体表现分化。中国经济顶住外部压力、克服内部困难，呈现回升向好态势，国内生产总值增长 5.2%，经济结构进一步优化，内循环主导作用明显增强，重大创新成果持续涌现，新旧动能转换加快，高质量发展扎实推进。积极的财政政策和稳健的货币政策协调配合，形成共促高质量发展合力。延续和优化相关税费政策，支持经营主体纾困

2024 -> PDF | 字符数 : 57385 | 输出 :
annual_reports\data\mdna_full_3\2024_mdna_full_3.txt

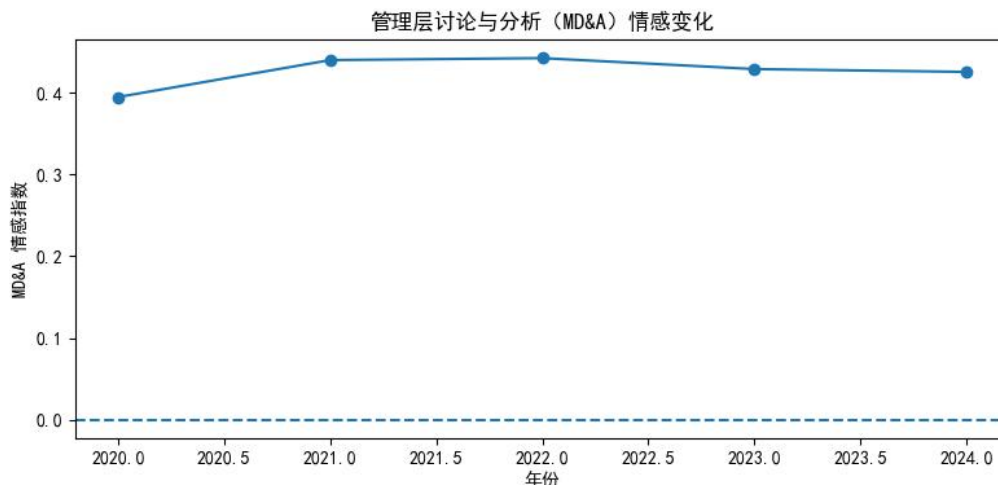
前 200 字预览：管理层讨论与分析 一、经济金融形势 2024 年，全球经济持续缓慢复苏，各主要经济体表现分化。中国经济顶住外部压力、克服内部困难， 向好态势不减，国内生产总值增长 5%。经济结构进一步优化，内循环主导作用明显增强，重大创新成果持续涌现，新旧动能转换加快，高质量发展扎实推进。积极的财政政策和稳健的货币政策协调配合，形成共促高质量发展合力。延续和优化相关税费政策， 支持经营主体纾困发展，扩大国内有效

年度 MD&A 情感结果：

	year	sentiment	pos_count	neg_count	mdna_length	pos_per_10k \
0	2020	0.394600	1343	583	74476	180.326548
1	2021	0.439848	1514	589	77939	194.254481
2	2022	0.442264	1580	611	78031	202.483628
3	2023	0.428864	1396	558	61034	228.724973
4	2024	0.425403	1414	570	57385	246.405855

	neg_per_10k	neg_ratio
0	78.280251	0.302700
1	75.571922	0.280076
2	78.302213	0.278868
3	91.424452	0.285568
4	99.329093	0.287298

```
# 情感指数折线图
plt.figure(figsize=(8, 4))
plt.plot(sentiment_df["year"], sentiment_df["sentiment"],
marker="o")
plt.axhline(0, linestyle="--")
plt.xlabel("年份")
plt.ylabel("MD&A 情感指数")
plt.title("管理层讨论与分析（MD&A）情感变化")
plt.tight_layout()
plt.show()
```



六、交通银行整体分析

1.交通银行词云图发展解读。

结合词云图关键词，交通银行 2020-2024 年战略脉络：以“服务实体经济、践行国有大行担当”为根基，以“一四五”战略为指引，聚焦“长三角龙头银行”牵引作用，深耕普惠金融、贸易金融、科技金融、财富金融四大特色业务，后围绕国家“五篇大文章”优化延伸为科技金融、绿色金融、普惠金融、养老金融、数字金融协同发展的战略布局，同时依托国际化布局强化跨境金融服务优势，以数字化转型赋能全业务链条，实现规模、质量、效益协同提升。

在战略实施过程中，交行始扣政策导向，将区域战略与全国布局相结合，重点强化长三角、粤港澳大湾区、京津冀等核心区域的业务渗透；将特色业务与国家战略深度绑定，在“一带一路”、乡村振兴、绿色转型等领域持续发力；将数字化转型作为战略突破口，通过技术赋能优化风险管控、提升服务效率，构建 G-B-C 多元联动的生态服务体系。

2.从 2020 到 2024 年交通银行发展数据盘点。

一是战略落地背景的宏观关联。2020-2024 年，宏观经济呈现“疫情冲击后复苏、结构调整深化、政策导向明确”的整体态势。期间，国家持续推进金融供给侧结构性改革，出台稳经济一揽子政策，重点支持“两新”“两重”领域、绿色转型、科技创新、乡村振兴等方向，为交通银行战略落地提供了明确政策指引。同时，适度宽松的货币政策为银行优化负债结构、降低资金成本创造了条件，而资本市场双向开放、跨境资本流动活跃的宏观环境，则助力交通银行发挥国际化优势，深化跨境金融服务。

区域宏观层面，长三角、粤港澳大湾区等核心区域经济持续领跑，为交行深耕区域市场提供了坚实经济基础，这也是三大区域贷款余额占比持续提升至 54.01%的核心动因。此外，人口老龄化趋势推动养老产业兴起、居民财富管理需求升级，均与交通银行养老金融体系构建、财富金融特色强化的战略方向高度契合。

二是规模稳步扩张，质量效益稳中提质。2020-2024 年，交行财务数据呈现“规模持续增长、盈利韧性凸显、资产质量稳定”的良好态势。资产规模合计从 2020 年末 106,976.16 亿元增长至 2024 年末 149,007.17 亿元，五年间增幅 39.30%，体现了业务扩张的稳健性。盈利水平保持稳定，营业总收入稳定在 2400-2600 亿元区间，净利润从 2020 年 795.70 亿元增至 2024 年 942.29 亿元，五年累计增长 18.42%，归母净利润同步增长至 935.86 亿元，基本每股收益从 0.99 元提升至

1.16 元，盈利增长韧性显著。资产质量优化，2024 年末不良贷款率 1.31%，较 2020 年显著下降。特色业务贡献凸显：理财与代理业务合计贡献 27.5% 中收份额；养老产业累计上线 323 款适老化财富管理产品；“交政通”接入 2000 余项政务服务，医疗付费一件事覆盖 97 个城市、1198 家医院，贸易融资发生额同比增长 31.76%，特色业务成为增长重要引擎。

三是股票估值低位企稳，长期投资价值凸显。2020-2024 年，交通银行 A 股（601328）股票呈现“低位波动、估值稳定、分红可观”特征。股价从 2020 年 3 月的阶段性低点 5.22 元逐步波动回升，2024 年末收盘价为 7.77 元，期间最高价接近 8 元，整体呈现稳步上行态势。市盈率维持在 5.7-6.3 倍区间，反映市场对银行股整体估值的审慎态度。分红回报稳定可观，2020-2024 年累计分红约每股 1.8 元，累计股息收益率超 23%。

四是情感分析正面评价为主，关注局部风险。呈现“整体正面、局部负面”格局。正面情感主导：市场对交行正面评价集中在战略落地成效、特色业务服务、社会责任履行等方面。投资者对其稳定的分红能力和低估值的投资价值给予正面评价。局部负面聚焦：主要集中在业务合规管理和客户服务细节方面。

3.整体总结与战略成效关联分析。

2020-2024 年，交通银行围绕战略稳步推进，数据表现与战略落地成效高度契合。宏观层面，政策导向与经济结构调整战略实施提供有利环境，推动银行在核心区域、重点

领域的业务布局持续深化；财务层面，资产规模扩张、盈利稳定增长、资产质量优化，印证战略导向下业务发展的质量与效益；股票层面，低估值、稳分红的特征，反映出市场对公司稳健经营的认可，也凸显了长期投资价值；情感层面，正面评价主导体现了战略落地的社会价值与市场认可，局部负面舆情则为后续优化合规管理、提升客户服务质量指明了方向。

综上所述：交通银行在 2020—2024 的五年间，成功实现“战略迭代升级、业务结构优化、经营质效提升”发展目标，践行了国有大行服务国家战略的使命担当，夯实了特色化、数字化的核心竞争优势，为持续发展奠定了坚实基础。